

Groupe 1



Onitama

Présentation du jeu

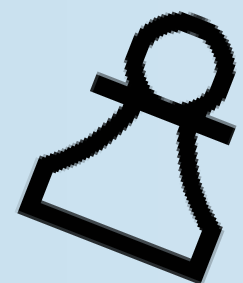
Comment jouer, déroulement et victoire

Fonctionnalités + IHM

Fonctionnalités illustrées par leur implémentation au niveau de l'IHM, IHM justifiée par la même occasion

IA

Principe, qualité, heuristique, ...



Présentation du Jeu



Onitama :

- Terrain de 5x5 cases
- 5 Pions par joueur (4 Élèves et 1 Maître)
- 5 Cartes uniques aléatoires par partie
 - Dont 2 par joueur (carte jouables)
 - Dont 1 en plus
(non jouable, défini qui commence, coin **bleu** = J1, **rouge** = J2)
 - Indique les déplacements pour un pion (case noire = pion choisi, case colorée = déplacements valides relatifs à case noire)
- Le pion choisi se déplace par rapport à la carte choisie
- Carte jouée échangée avec la carte en plus
- Victoire
 - Le pion maître adverse est capturé par n'importe quel pion
 - Le joueur courant place son pion maître sur la case temple (position initiale des maîtres en début de partie)





Fonctionnalités + IHM



Fonctionnement du moteur

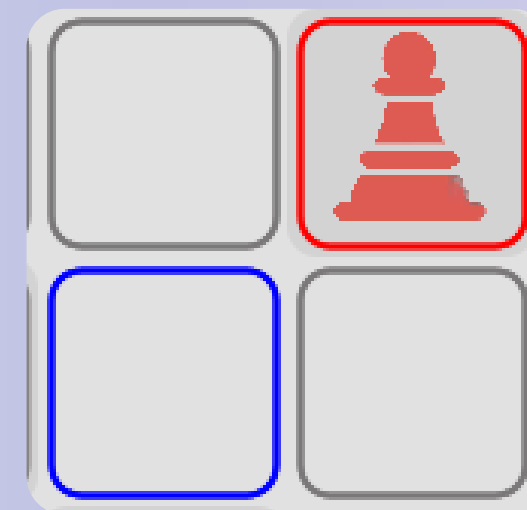
Le moteur gère l'état de la grille et les pions que possèdent chaque joueur, ainsi que les cartes en rotation.

Il gère les coups possibles en fonction du tour, de la carte, et du pion sélectionné

Jouer un coup

Le moteur utilise un automate pour gérer l'état de la grille (État par défaut, pion sélectionné, jouer le coup) et vérifie si le coup qu'on souhaite jouer est valide

Jouer un coup



↶ Annuler / Refaire ↷

Usage d'un historique
(2 piles)



Restaurer le plateau
(carte échangée, pion
capturé, joueur courant, ...)

↶ Annuler / Refaire ↷

Bouton refaire



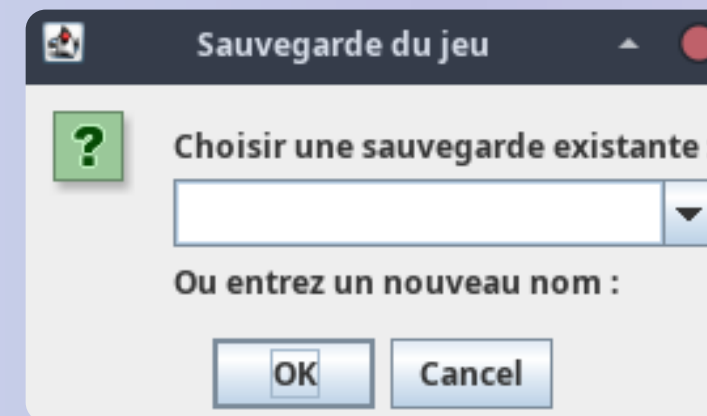
Bouton annuler

(Grisés quand action impossible)

 **Sauvegarder / Charger** 
une partie

Le moteur écrit / lit l'état du jeu
(type joueur, état joueurs,
historique, état grille) dans un
fichier

 Sauvegarder / Charger 
une partie



REPRENDRE

sauvegarde.sav



Nouvelle partie *

Le moteur réinitialise l'état du jeu,
l'interface affiche le nouvel état du
jeu

Nouvelle partie ✱



NOUVELLE PARTIE

Suggérer un coup

Utilise l'IA de niveau MOYEN pour calculer un coup pertinent à suggérer au joueur courant (HUMAIN) en cas de difficulté

Suggérer un coup 



(Grisé quand action impossible)

Choix du niveau d'une instance d'IA 

FAIBLE

MOYEN

DIFFICILE

Choix du niveau d'une instance d'IA 🤖

Joueur 1

Humain

▼

Joueur 2

IA - Difficile

▼

Fonctionnalités + IHM

MODES

 Humain **vs**  Humain

 Humain **vs**  IA

 IA **vs**  IA

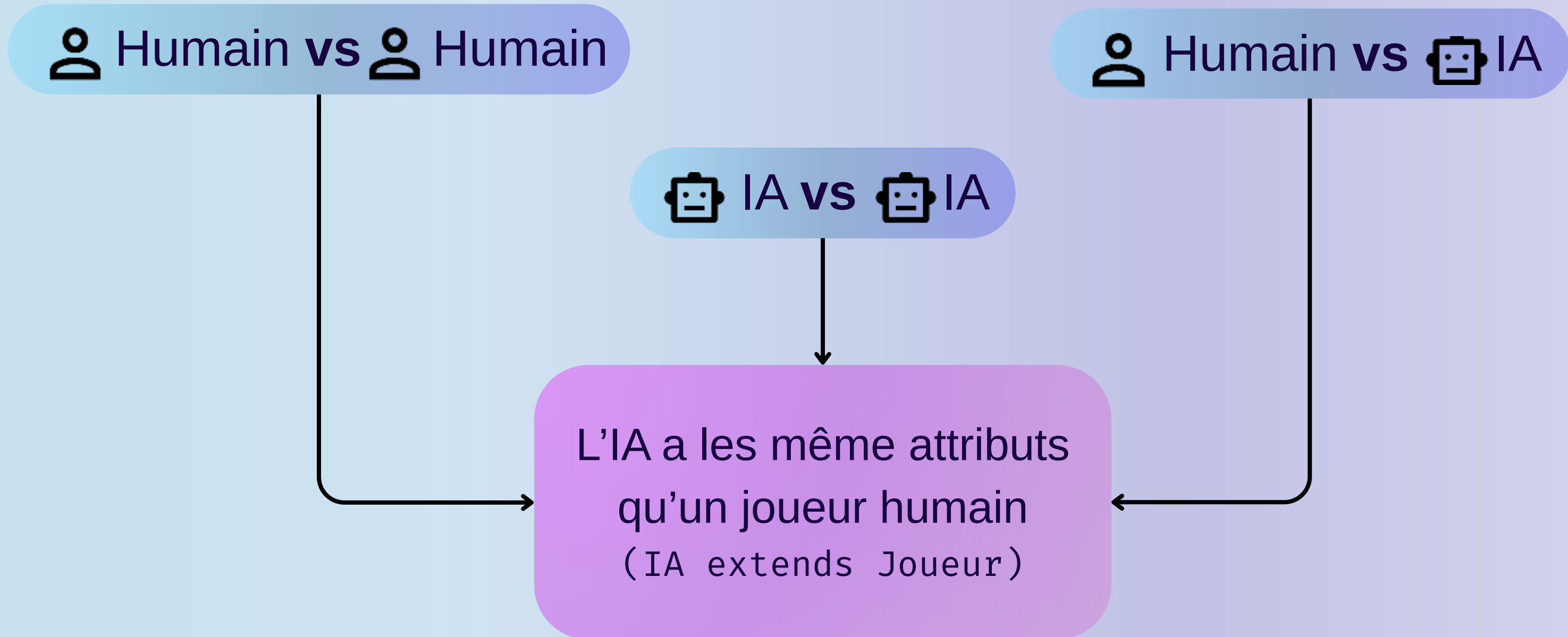
MODES

 Humain vs  Humain

 Humain vs  IA

 IA vs  IA

L'IA a les même attributs
qu'un joueur humain
(IA extends Joueur)



 Humain **vs**  Humain

 IA **vs**  IA

 Humain **vs**  IA

Usage d'un automate pour
gérer l'état du jeu et faire
jouer l'IA (ou les IA)

Fonctionnalités + IHM

 Humain **vs**  Humain

 IA **vs**  IA

 Humain **vs**  IA

joueur 1

Humain

▼

joueur 2

Humain

▼



joueur 1

Humain

▼

joueur 2

IA - Difficile

▼

IHM - Validation utilisateurs

Retours utilisateurs, clarté,
ergonomie



IA



Stratégie

L'IA cherche à savoir quel coup lui permet d'arriver à terme dans un état du jeu qui lui est favorable



Nécessite de calculer plusieurs coups (qui donnent des configurations nouvelles) à l'avance

Arbre MiniMax



Noeud MAX

IA

7



Noeud MIN

4

7

1

5

4

7

1

4

2

2

5

7

4

Au tour de :

Adversaire

IA

Adversaire

Coût de l'évaluation

Complexité de parcours de l'arbre : $O(b^p)$ avec b le facteur de branchement (nombre d'états successeurs) et p la profondeur

On doit limiter la profondeur d'exploration (horizon fini)

Concernant Onitama, le facteur de branchement est 40 dans le pire des cas.

Comme

$$40^2 = 1\,600$$

$$40^3 = 64\,000$$

$$40^4 = 2\,560\,000$$

...

$$40^6 = 4\,096\,000\,000$$

On ne peut pas de manière réaliste atteindre toutes les configurations finales pour les évaluer.

Méthode d'évaluation

Évaluation d'une configuration non terminale à l'aide d'une heuristique composée de plusieurs sous-heuristiques (somme pondérée des sous-heuristiques)

Méthode d'évaluation

Heuristique :

$$\begin{aligned} & 100 * \text{nbPions}(\text{etatJeu}) \\ & + \text{valeurCartes}(\text{etatJeu}) \\ & + 2 * \text{successeursNombreCaptures}(\text{etatJeu}) \\ & + -2 * \text{distancePionsCourantMaitreAdverse}(\text{etatJeu}) \\ & + 2 * \text{distancePionsAdverseMaitreCourant}(\text{etatJeu}) \\ & + -1.5 * \text{distanceMaitreAdverseTempleCourant}(\text{etatJeu}) \\ & + 1.5 * \text{distanceMaitreCourantTempleAdverse}(\text{etatJeu}) \\ & + \text{victoireDefaite}(\text{idJoueur}, \text{etatJeu}) \end{aligned}$$

On peut faire varier la pondération pour trouver la formule la plus efficace

Améliorations possibles

On peut également stocker les états déjà rencontrés sous forme de vecteur de bits pour connaître à l'avance leurs successeurs et/ou leur valeur

Validation et performances obtenues

L'IA difficile bat l'IA moyenne dans la majorité des cas, qui elle même bat l'IA facile dans la majorité des cas

Démo Jeu